



UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

COMPUTAÇÃO DE ALTO DESEMPENHO

DOCENTE: CARLA DOS SANTOS SANTANA

E SAMUEL XAVIER DE SOUZA

DISCENTE: CLARA VIRGÍNIA NASCIMENTO SANTOS (20240002212)

Tarefa 15

1. ENUNCIADO

Implemente uma simulação da difusão de calor em uma barra 1D, dividida entre dois ou mais processos MPI. Cada processo deve simular um trecho da barra com células extras para troca de bordas com vizinhos. Implemente três versões: uma com MPI_Send/MPI_Recv, outra com MPI_Isend/MPI_Irecv e MPI_Wait, e uma terceira usando MPI_Test para atualizar os pontos internos enquanto aguarda a comunicação. Compare os tempos de execução e discuta os ganhos com sobreposição de comunicação e computação.

2. DISCUSSÃO

Nas análises anteriores, foi utilizada a biblioteca OpenMP, na qual todas as threads compartilhavam a mesma memória, isto é, um modelo de memória compartilhada. No entanto, nesta tarefa será usado o MPI (Message Passing Interface), que segue um modelo diferente, o de memória distribuída. Dessa maneira, ao invés de threads dividindo um mesmo espaço, cada processo possui sua própria memória isolada, e a única forma de um processo conhecer os dados do outro é por meio de troca explícita de mensagens.

2.1. Difusão de calor em 1D

A difusão de calor descreve como a temperatura se espalha ao longo de um material com o passar do tempo. Quando essa barra é representada no computador, ela é dividida em vários pontos, em que cada ponto guarda a temperatura naquele trecho. Assim, a temperatura

nova de cada ponto depende dela mesma e dos seus dois vizinhos (esquerdo e direito), conforme a fórmula discretizada abaixo:

$$\text{novo}[i] = \text{atual}[i] + \alpha * (\text{atual}[i-1] - 2*\text{atual}[i] + \text{atual}[i+1])$$

Em relação ao fator α (alpha), ele controla a velocidade com que o calor se espalha. Contudo, para que a simulação não fique instável e "exploda" com valores absurdos, esse fator precisa ser menor ou igual a 0.5.

2.2. Decomposição de domínio e células fantasma

Já que cada processo possui memória própria, a barra inteira é repartida em pedaços, de tal forma que cada processo fica responsável por simular apenas o seu trecho. Esse procedimento é chamado de decomposição de domínio. Todavia, surge um problema: para calcular o ponto que fica bem na ponta do trecho, é necessário saber o valor do vizinho que está no processo ao lado. Para resolver isso, cada processo aloca duas posições extras, uma em cada borda, chamadas de células fantasma (ou halo). Dessa forma, antes de cada cálculo, os processos trocam entre si os valores das bordas e os guardam nessas células fantasma, simulando o "vizinho" que mora no outro processo.

NOTA: para os processos das pontas da barra, que não têm vizinho de um dos lados, foi usado o MPI_PROC_NULL. Assim, a tentativa de troca para esse "vizinho inexistente" simplesmente não faz nada, evitando erros.

2.3. Comunicação bloqueante: MPI_Send e MPI_Recv

As funções MPI_Send e MPI_Recv realizam a troca de mensagens de forma bloqueante, ou seja, o processo fica parado esperando a comunicação acabar antes de seguir adiante. Por consequência, o cálculo só começa depois que toda a troca de bordas terminou. Vale ressaltar que, como cada processo troca apenas um valor com cada vizinho, a mensagem é pequena o bastante para o MPI conseguir guardá-la temporariamente, evitando que o programa trave (deadlock).

2.4. Comunicação não-bloqueante: MPI_Isend, MPI_Irecv e MPI_Wait

Por outro lado, as funções `MPI_Isend` e `MPI_Irecv` são não-bloqueantes, isto é, elas apenas "disparam" a comunicação e devolvem o controle imediatamente, sem travar o processo. Dessa maneira, é possível disparar todos os envios e recebimentos de uma vez. Após isso, é usado o `MPI_Wait` (ou `MPI_Waitall`) para garantir que todas as mensagens realmente chegaram antes de usar os dados. Além disso, essa abordagem já elimina o risco de deadlock naturalmente, sem precisar do truque par/ímpar.

2.5. Sobreposição de comunicação e computação: `MPI_Test`

A terceira versão aproveita a ideia de que parte do trabalho não depende dos vizinhos. Enquanto a comunicação acontece em segundo plano, os pontos internos do trecho (todos, menos os dois das bordas) já podem ser calculados, pois eles não precisam das células fantasma. Para isso, é usado o `MPI_Test`, que apenas pergunta se a comunicação já terminou, sem ficar parado esperando. Dessa forma, o tempo gasto na troca de mensagens é "escondido" atrás do tempo de cálculo, o que é chamado de sobreposição de comunicação e computação. Logo após, somente os dois pontos da borda, que dependem dos vizinhos, são atualizados.

2.6. Compilação e execução com MPI

A compilação utiliza o compilador `mpicc` (ou o `gcc` com as bibliotecas do MPI), e a execução é feita pelo `mpiexec`/`mpirun`, no qual a flag `-n` indica quantos processos serão criados:

```
mpiexec -n 4 tarefa15.exe
```

2.7. Etapas Para Resolução Da Tarefa

- 1º) Montar a fórmula discretizada da difusão de calor;
- 2º) Dividir a barra entre os processos com células fantasma (decomposição de domínio);
- 3º) Implementar a Versão 1 com `MPI_Send`/`MPI_Recv`;
- 4º) Implementar a Versão 2 com `MPI_Isend`/`MPI_Irecv` + `MPI_Wait`;
- 5º) Implementar a Versão 3 com `MPI_Test` e sobreposição;

- 6º) Conferir se as três versões chegam ao mesmo resultado físico;
- 7º) Medir e comparar os tempos de execução;
- 8º) Variar o número de processos.

2.8. RESOLUÇÃO EM CÓDIGO DA TAREFA

Segue no apêndice.

2.9. OUTPUT

Segue no apêndice.

3. CONCLUSÃO

Os resultados demonstram, de forma clara, que a escolha da estratégia de comunicação tem impacto direto no desempenho, e que mais sofisticação nem sempre significa mais velocidade. No caso desta simulação 1D, em que cada processo troca apenas 1 double com seus vizinhos, a Versão 1 (bloqueante) foi a mais rápida justamente por ter o menor overhead de API, já que a mensagem é tão pequena que o MPI_Send consegue bufferizá-la e retornar quase imediatamente, sem custo de latência perceptível.

A Versão 2, embora conceitualmente não-bloqueante, foi a mais lenta porque o custo de gerenciar as requisições assíncronas superou em muito o benefício, dado que não havia latência real a esconder. A Versão 3 ficou numa posição intermediária, aproveitando parcialmente a sobreposição ao calcular os pontos internos durante a comunicação, porém ainda sofrendo com o mesmo overhead da API não-bloqueante.

Portanto, é possível concluir que a sobreposição de comunicação e computação é uma técnica poderosa, mas seu benefício depende diretamente da relação entre o volume de dados comunicados e o volume de cálculo. Assim, para problemas com halo maior (como simulações 2D e 3D), as Versões 2 e 3 tenderiam a superar a Versão 1, que é justamente o cenário real de aplicações de alto desempenho.

4. APÊNDICE

4.1. RESOLUÇÃO

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

#define N_GLOBAL 1000000 // numero total de pontos da barra
#define PASSOS 1000 // quantos passos de tempo a simulacao roda
#define ALPHA 0.1 // velocidade da difusao (tem que ser <= 0.5)

// VERSAO 1: MPI_Send / MPI_Recv
void versao_1() {
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank); // numero desse processo
    MPI_Comm_size(MPI_COMM_WORLD, &size); // total de processos

    int local_n = N_GLOBAL / size; // pedaco da barra que esse processo
cuida

    // descobrir os vizinhos
    int viz_esq = rank - 1;
    int viz_dir = rank + 1;
    if (rank == 0) viz_esq = MPI_PROC_NULL; // o primeiro nao tem vizinho a
esquerda
    if (rank == size - 1) viz_dir = MPI_PROC_NULL; // o ultimo nao tem
vizinho a direita

    // 2 posicoes a mais: uma celula fantasma em cada ponta
    double *atual = malloc((local_n + 2) * sizeof(double));
    double *novo = malloc((local_n + 2) * sizeof(double));

    // comeca tudo frio (zero)
    for (int i = 0; i < local_n + 2; i++) { atual[i] = 0; novo[i] = 0; }
    if (rank == 0) atual[1] = 100; // ponta esquerda quente

    MPI_Barrier(MPI_COMM_WORLD); // todo mundo comeca junto
    double inicio = MPI_Wtime();

    for (int passo = 0; passo < PASSOS; passo++) {
        // troca as bordas com os vizinhos (mensagem pequena, nao trava)
        MPI_Send(&atual[local_n], 1, MPI_DOUBLE, viz_dir, 0,
MPI_COMM_WORLD);
        MPI_Recv(&atual[0], 1, MPI_DOUBLE, viz_esq, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
        MPI_Send(&atual[1], 1, MPI_DOUBLE, viz_esq, 0, MPI_COMM_WORLD);
        MPI_Recv(&atual[local_n+1], 1, MPI_DOUBLE, viz_dir, 0,
MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

```

        // calcula a temperatura nova de cada ponto
        for (int i = 1; i <= local_n; i++) {
            novo[i] = atual[i] + ALPHA * (atual[i-1] - 2*atual[i] +
atual[i+1]);
        }

        // mantem as pontas da barra fixas
        if (rank == 0) novo[1] = 100;
        if (rank == size - 1) novo[local_n] = 0;

        // copia o "novo" pro "atual" pro proximo passo
        for (int i = 1; i <= local_n; i++) {
            atual[i] = novo[i];
        }
    }

    double fim = MPI_Wtime();

    // soma todo o calor (so pra conferir se as 3 versoes batem)
    double soma_local = 0, soma_total = 0;
    for (int i = 1; i <= local_n; i++) soma_local += atual[i];
    MPI_Reduce(&soma_local, &soma_total, 1, MPI_DOUBLE, MPI_SUM, 0,
MPI_COMM_WORLD);

    if (rank == 0)
        printf("Versao 1 (Send/Recv) -> Tempo: %f s | Calor: %f\n", fim -
inicio, soma_total);

    free(atual); free(novo);
}

// VERSAO 2: MPI_Isend / MPI_Irecv + MPI_Wait
void versao_2() {
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int local_n = N_GLOBAL / size;
    int viz_esq = rank - 1;
    int viz_dir = rank + 1;
    if (rank == 0) viz_esq = MPI_PROC_NULL;
    if (rank == size - 1) viz_dir = MPI_PROC_NULL;

    double *atual = malloc((local_n + 2) * sizeof(double));
    double *novo = malloc((local_n + 2) * sizeof(double));
    for (int i = 0; i < local_n + 2; i++) { atual[i] = 0; novo[i] = 0; }
    if (rank == 0) atual[1] = 100;

```

```

MPI_Barrier(MPI_COMM_WORLD);
double inicio = MPI_Wtime();

for (int passo = 0; passo < PASSOS; passo++) {
    MPI_Request reqs[4];
    // dispara as 4 comunicacoes (nao trava o programa)
    MPI_Irecv(&atual[0], 1, MPI_DOUBLE, viz_esq, 0, MPI_COMM_WORLD,
&reqs[0]);
    MPI_Irecv(&atual[local_n+1], 1, MPI_DOUBLE, viz_dir, 0,
MPI_COMM_WORLD, &reqs[1]);
    MPI_Isend(&atual[1], 1, MPI_DOUBLE, viz_esq, 0, MPI_COMM_WORLD,
&reqs[2]);
    MPI_Isend(&atual[local_n], 1, MPI_DOUBLE, viz_dir, 0,
MPI_COMM_WORLD, &reqs[3]);

    // espera as 4 terminarem antes de calcular
    MPI_Waitall(4, reqs, MPI_STATUSES_IGNORE);

    for (int i = 1; i <= local_n; i++) {
        novo[i] = atual[i] + ALPHA * (atual[i-1] - 2*atual[i] +
atual[i+1]);
    }
    if (rank == 0) novo[1] = 100;
    if (rank == size - 1) novo[local_n] = 0;
    for (int i = 1; i <= local_n; i++) {
        atual[i] = novo[i];
    }
}

double fim = MPI_Wtime();
double soma_local = 0, soma_total = 0;
for (int i = 1; i <= local_n; i++) soma_local += atual[i];
MPI_Reduce(&soma_local, &soma_total, 1, MPI_DOUBLE, MPI_SUM, 0,
MPI_COMM_WORLD);
if (rank == 0)
    printf("Versao 2 (Isend/Irecv+Wait) -> Tempo: %f s | Calor: %f\n",
fim - inicio, soma_total);
free(atual); free(novo);
}

// VERSAO 3: MPI_Test (calcula enquanto comunica)
void versao_3() {
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int local_n = N_GLOBAL / size;
    int viz_esq = rank - 1;

```

```

    int viz_dir = rank + 1;
    if (rank == 0) viz_esq = MPI_PROC_NULL;
    if (rank == size - 1) viz_dir = MPI_PROC_NULL;

    double *atual = malloc((local_n + 2) * sizeof(double));
    double *novo = malloc((local_n + 2) * sizeof(double));
    for (int i = 0; i < local_n + 2; i++) { atual[i] = 0; novo[i] = 0; }
    if (rank == 0) atual[1] = 100;

    MPI_Barrier(MPI_COMM_WORLD);
    double inicio = MPI_Wtime();

    for (int passo = 0; passo < PASSOS; passo++) {
        MPI_Request reqs[4];
        MPI_Irecv(&atual[0], 1, MPI_DOUBLE, viz_esq, 0, MPI_COMM_WORLD,
        &reqs[0]);
        MPI_Irecv(&atual[local_n+1], 1, MPI_DOUBLE, viz_dir, 0,
        MPI_COMM_WORLD, &reqs[1]);
        MPI_Isend(&atual[1], 1, MPI_DOUBLE, viz_esq, 0, MPI_COMM_WORLD,
        &reqs[2]);
        MPI_Isend(&atual[local_n], 1, MPI_DOUBLE, viz_dir, 0,
        MPI_COMM_WORLD, &reqs[3]);

        // enquanto a comunicacao acontece, ja calculo os pontos do MEIO
        // nao preciso esperar)
        for (int i = 2; i <= local_n - 1; i++) {
            novo[i] = atual[i] + ALPHA * (atual[i-1] - 2*atual[i] +
atual[i+1]);
        }

        // fico perguntando se a comunicacao ja terminou
        int terminou = 0;
        while (!terminou) {
            MPI_Testall(4, reqs, &terminou, MPI_STATUSES_IGNORE);
        }

        // agora que as bordas chegaram, calculo os 2 pontos das pontas
        novo[1] = atual[1] + ALPHA * (atual[0] - 2*atual[1] + atual[2]);
        novo[local_n] = atual[local_n] + ALPHA * (atual[local_n-1] -
2*atual[local_n] + atual[local_n+1]);

        if (rank == 0) novo[1] = 100;
        if (rank == size - 1) novo[local_n] = 0;
        for (int i = 1; i <= local_n; i++) {
            atual[i] = novo[i];
        }
    }
}

```



```

    double fim = MPI_Wtime();
    double soma_local = 0, soma_total = 0;
    for (int i = 1; i <= local_n; i++) soma_local += atual[i];
    MPI_Reduce(&soma_local, &soma_total, 1, MPI_DOUBLE, MPI_SUM, 0,
MPI_COMM_WORLD);
    if (rank == 0)
        printf("Versao 3 (Test/sobreposicao) -> Tempo: %f s | Calor: %f\n",
fim - inicio, soma_total);
    free(atual); free(novo);
}

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (rank == 0) printf("Rodando com %d processos\n", size);

    versao_1();
    versao_2();
    versao_3();

    MPI_Finalize();
    return 0;
}

```

4.2. OUTPUT

```

PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa15> mpiexec -n 4 tarefa15.exe
Rodando com 4 processos
Versao 1 (Send/Recv) -> Tempo: 0.766543 s | Calor: 1179.225416
Versao 2 (Isend/Irecv+Wait) -> Tempo: 1.876344 s | Calor: 1179.225416
Versao 3 (Test/sobreposicao) -> Tempo: 0.837732 s | Calor: 1179.225416
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa15> |

```

